AWS CLOUD NETWORKING

AWS VPC Peering Project

Private connectivity between two VPCs, bastion access, and a controlled ICMP failure-and-recovery test

Field	Detail
Student	Rudra Sanjay Rakshe
Course	AWS Cloud Networking
AWS Region	Asia Pacific (Sydney), `ap-southeast-2`
Evidence captured	28 October 2026 (from screenshot timestamps; no separate completion date was supplied)
Repository	https://github.com/pawaraditya0903/AWS-VPC-Peering-Project
Report file	RudraRakshe_EWB_VPC_Peering.pdf

Table of Contents

1. Executive summary....2
2. Objectives and requirements....2
3. Architecture....2
3.1 Key concepts....3
4. Resource inventory....4
5. Routing configuration....4
5.1 Forward and return path....5
5.2 Internet access to the bastion....5
6. Security configuration....5
7. Test results and evidence....6
7.1 Test 1: Baseline failure (before peering)....6
7.2 Test 2: Forward path (App-A to App-B)....6
7.3 Test 3: Reverse path (App-B to App-A)....7
7.4 Test 4: Private SSH through the bastion....7
7.5 Test 5: Isolation (no public IPv4)....8
8. Controlled ICMP failure and recovery....9
8.1 Observed results....10
8.2 Limits of this evidence....11
8.3 What this demonstrates....11
9. Troubleshooting and lessons learned....11
Lessons learned....11
10. Cleanup evidence and cost considerations....12
11. Conclusion....14
12. Appendix....14
Appendix A: Missing evidence and checklist....14
Appendix B: Supplementary file test1.png....14
Appendix C: Evidence file map....15

1. Executive summary

This project connects two Amazon VPCs in the Sydney region (`ap-southeast-2`) with a VPC peering connection, and proves private communication between one EC2 instance in each VPC. A public bastion host in VPC A is the only internet-facing entry point; the two application instances (App-A and App-B) have no public IPv4 address and are reached by SSH through the bastion using their private addresses.

The screenshots supplied support the following verified outcomes. Where evidence is missing or limited, the table says so rather than claiming a pass.

Test	Status	What the evidence shows
Test 1: Baseline failure (before peering)	Not verified. Evidence not provided	No capture of a failed ping before peering was supplied. The file `test1.png` shows a successful ping with peering already active, so it cannot serve as a baseline (see section 8.1).
Test 2: Forward path (App-A to App-B)	Passed	Figure 2: 4 packets transmitted, 4 received, 0% loss (two consecutive runs).
Test 3: Reverse path (App-B to App-A)	Passed	Figure 3: 4 packets transmitted, 4 received, 0% loss.
Test 4: Private SSH through the bastion	Passed	Figure 4: SSH to `10.20.1.10` via the bastion; hostname and IP confirmed. The command header in Figure 2 shows the same method for App-A (`10.10.2.10`).
Test 5: Isolation (no public IPv4)	Passed	Figure 5 and Figure 6: Public IPv4 address and Public DNS are blank for App-B and App-A.
Controlled ICMP failure and recovery	Partial. Ping results shown; rule edits not captured	Figure 7: 100% packet loss. Figure 8: 0% loss after recovery. No console screenshot of the security-group rule removal or restoration was supplied.
Cleanup	Partial	Instances terminated (Figure 9), no EBS volumes remaining (Figure 10), peering connection deleted (Figure 11). Other resources are not evidenced (section 10).

2. Objectives and requirements

The objectives below come from the project brief as summarised in the report request. The original project instructions PDF was not available when this report was prepared, so test names and numbers follow the summary supplied, and the PDF's submission checklist could not be checked directly.

- Connect two VPCs with non-overlapping CIDR ranges through an active VPC peering connection.
- Run one public bastion EC2 instance in VPC A and one private EC2 instance in each VPC.
- Reach the private instances from a Windows laptop by SSH through the bastion.
- Demonstrate private communication in both directions with ping (Tests 2 and 3), private SSH (Test 4) and isolation from the internet (Test 5).
- Show that security-group rules control traffic by removing and restoring an ICMP rule.
- Clean up billable resources and record the evidence.

3. Architecture

VPC A (`ewb-vpc-a`, `10.10.0.0/16`) holds a public subnet (`ewb-a-public`, `10.10.1.0/24`) with the bastion host and a private subnet (`ewb-a-private`, `10.10.2.0/24`) with App-A. An internet gateway (`ewb-igw-a`) is attached to VPC A. VPC B (`ewb-vpc-b`, `10.20.0.0/16`) holds one private

subnet (`ewb-b-private`, `10.20.1.0/24`) with App-B. The two VPCs are joined by the peering connection `pcx-04950eee78a68adf1`.

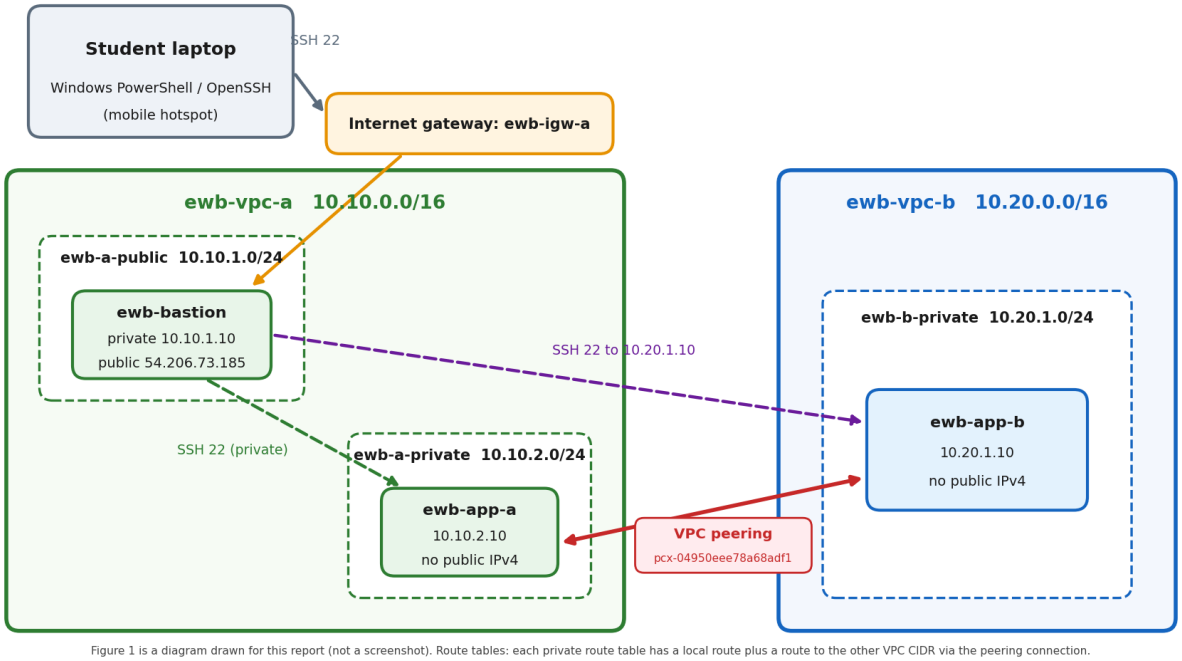


Figure 1: *Architecture of the project with the actual CIDR ranges and private IPs (diagram drawn for this report).*

3.1 Key concepts

Concept	Explanation
Non-overlapping CIDRs	A route table chooses a destination by matching an address range. If both VPCs used the same range, a packet for `10.10.2.10` could not be told apart from a local address, so AWS refuses to peer VPCs with overlapping CIDRs. Here `10.10.0.0/16` and `10.20.0.0/16` are distinct.
Peering alone is not enough	A peering connection only creates a possible path. Traffic does not flow until each side has a route that sends the other VPC's CIDR to the peering connection, and security groups and network ACLs allow the traffic.
Routes on both sides	A ping needs a request path and a reply path. Each private route table sends the other VPC's CIDR to the peering connection; if one side lacked its route, requests would arrive but replies would be lost.
Security groups	Stateful, instance-level filters. Rules are matched on traffic initiated toward the instance; replies to connections the instance started are allowed automatically. This explains why removing an inbound ICMP rule affects only pings started by the other side.
Network ACLs	Stateless, subnet-level filters. The default ACLs in this project allow all inbound traffic (rule 100, `0.0.0.0/0`), so they do not block any test.
Private IPs for App-A and App-B	The application servers need no inbound internet access. Giving them private addresses only removes them from the public internet and shrinks the attack surface.
Bastion host	A hardened, internet-reachable instance that administrators SSH into first and then use as a stepping stone. The private instances accept SSH only from the bastion's private IP `10.10.1.10/32`.

4. Resource inventory

The last column states where each value is supported. "Screenshot" means it appears in a supplied screenshot. "Supplied text" means the student provided it in the project description but no screenshot of it was supplied. IDs that were not supplied are not listed.

Resource	Name	ID / value	Support
VPC A	`ewb-vpc-a`	`10.10.0.0/16` (VPC ID not supplied)	Supplied text
VPC B	`ewb-vpc-b`	`10.20.0.0/16` (VPC ID not supplied)	Supplied text
Peering VPC IDs	Names truncated in console	Requester `vpc-05d428ad6f23309a6`; accepter `vpc-08ee2f62ef6ca0958` (the report does not assign these to A or B)	Screenshot
Public subnet (A)	`ewb-a-public`	`10.10.1.0/24`, `subnet-0f2d077ab0f838d40`	Supplied text
Private subnet (A)	`ewb-a-private`	`10.10.2.0/24`	Supplied text
Private subnet (B)	`ewb-b-private`	`10.20.1.0/24`	Supplied text
Internet gateway	`ewb-igw-a`	`igw-0c9d444d4bf46e346`, attached to VPC A	Supplied text
Peering connection	`ewb-vpc-a-to-vpc-b`	`pcx-04950eee78a68adf1`; Active during testing, shown as Deleted after cleanup	Screenshot (deleted state); Supplied text (Active)
Route tables	`ewb-rt-public`, `ewb-rt-a-private`, `ewb-rt-b-private`	IDs not supplied	Supplied text
Security groups	`ewb-sg-bastion`, `ewb-sg-app-a`, `ewb-sg-app-b`	`ewb-sg-app-b` = `sg-005b2092a76af1dd5`; others not supplied	Supplied text
Bastion (final)	`ewb-bastion`	`i-044f460b5fb67b887`, private `10.10.1.10`, public `54.206.73.185`	Screenshot (Figure 9, command lines)
Bastion (original)	`ewb-bastion`	`i-0275c2cb17ef82f49`, terminated and replaced during troubleshooting	Screenshot (instance list)
App-A	`ewb-app-a`	`i-02d7cd88e7647e009`, private `10.10.2.10`, no public IPv4	Screenshot (Figure 6)
App-B	`ewb-app-b`	`i-01b723cce810423fd`, private `10.20.1.10`, no public IPv4	Screenshot (Figure 5)
Instance details	All three	t3.micro, Availability Zone `ap-southeast-2a`, Amazon Linux 2023 (login banner), key pair `ewb-key`	Screenshot

5. Routing configuration

The route tables below are taken from the supplied project description; no route-table console screenshots were provided. Functional evidence that both peering routes exist is that pings succeeded in both directions (Tests 2 and 3).

Route table	Routes	Associated subnet
ewb-rt-public (VPC A)	10.10.0.0/16 to local 0.0.0.0/0 to igw-0c9d444d4bf46e346 10.20.0.0/16 to pcx-04950eee78a68adf1	ewb-a-public
ewb-rt-a-private (VPC A)	10.10.0.0/16 to local 10.20.0.0/16 to the peering connection	ewb-a-private
ewb-rt-b-private (VPC B)	10.20.0.0/16 to local 10.10.0.0/16 to the peering connection	ewb-b-private

5.1 Forward and return path

1. **Forward (App-A to App-B):** App-A (10.10.2.10) sends an echo request to 10.20.1.10. ewb-rt-a-private matches 10.20.0.0/16 and forwards it to the peering connection, which delivers it into VPC B and on to App-B.
2. **Return (App-B to App-A):** App-B replies to 10.10.2.10. ewb-rt-b-private matches 10.10.0.0/16 and forwards the reply over the same peering connection back to App-A.

Both routes are needed: with only the forward route, App-B would receive the request but could not send a reply. The same logic applies in the reverse direction (Test 3).

5.2 Internet access to the bastion

ewb-rt-public sends 0.0.0.0/0 to the internet gateway and is associated with the public subnet, which gives the bastion a path to and from the internet through its public IPv4 address. The private subnets have no default route to the internet, which is why App-A and App-B are not internet-reachable.

6. Security configuration

Rules are documented from the supplied project description. No security-group console screenshots were supplied.

Security group	Inbound	Outbound / notes
ewb-sg-bastion (VPC A)	SSH, TCP 22, from the laptop's public IPv4 /32 (intended configuration)	All traffic allowed. See the note below about the temporary 0.0.0.0/0 source.
ewb-sg-app-a (VPC A)	SSH, TCP 22, from 10.10.1.10/32 (bastion) ICMP Echo Request from 10.20.1.10/32 (App-B)	Default allow-all
ewb-sg-app-b (VPC B) sg-005b2092a76af1dd5	SSH, TCP 22, from 10.10.1.10/32 (bastion) ICMP Echo Request from 10.10.2.10/32 (App-A)	Allow all to 0.0.0.0/0. This group was the only one attached to App-B.

Purpose of the bastion. Only the bastion has a public IPv4 address and an inbound SSH rule from the internet. The private instances accept SSH solely from the bastion's private IP, so administrators must pass through one controlled entry point.

Temporary exposure to disclose. The student worked from a mobile hotspot whose public IPv4 address changed between connections, so a /32 rule repeatedly stopped matching. To finish testing, the bastion's SSH source was temporarily set to 0.0.0.0/0 (open to the internet). This is insecure for

anything but a short test. The best practice is to set the source back to the current `/32` straight after connecting. No screenshot of the final state of this rule is included; the bastion was later terminated (Figure 9).

7. Test results and evidence

Terminal screenshots were cropped only to remove blank terminal space; the Windows taskbar clock was kept. The terminal output and console values are unaltered. Taskbar times are the time each screenshot was taken, in India Standard Time (UTC+5:30), while the shell's `Last login` and `date` lines are in UTC.

7.1 Test 1: Baseline failure (before peering)

Item	Detail
Objective	Show that App-A cannot reach App-B before the peering connection and routes exist.
Method	`ping -c 4 10.20.1.10` from App-A before peering was set up.
Expected	100% packet loss.
Observed	Evidence not provided. The only file supplied for this test, `test1.png`, shows a successful ping (4 of 4 received) with peering already active. It is reproduced in Appendix B for transparency.
Conclusion	Not verified. This report does not claim a pre-peering baseline. Test 2 shows connectivity with peering in place, but cannot by itself show it was absent beforehand.

7.2 Test 2: Forward path (App-A to App-B)

Item	Detail
Objective	Show App-A reaches App-B privately over the peering connection.
Method	SSH to App-A through the bastion, then run `ping -c 4 10.20.1.10`.
Expected	4 replies, 0% packet loss.
Observed	Figure 2 shows two consecutive runs, each with 4 transmitted, 4 received, 0% packet loss, round-trip about 0.16 to 0.28 ms and `ttl=127`.
Conclusion	Passed. The forward route, the return route, and the App-B inbound ICMP rule (source `10.10.2.10/32`) all worked together.

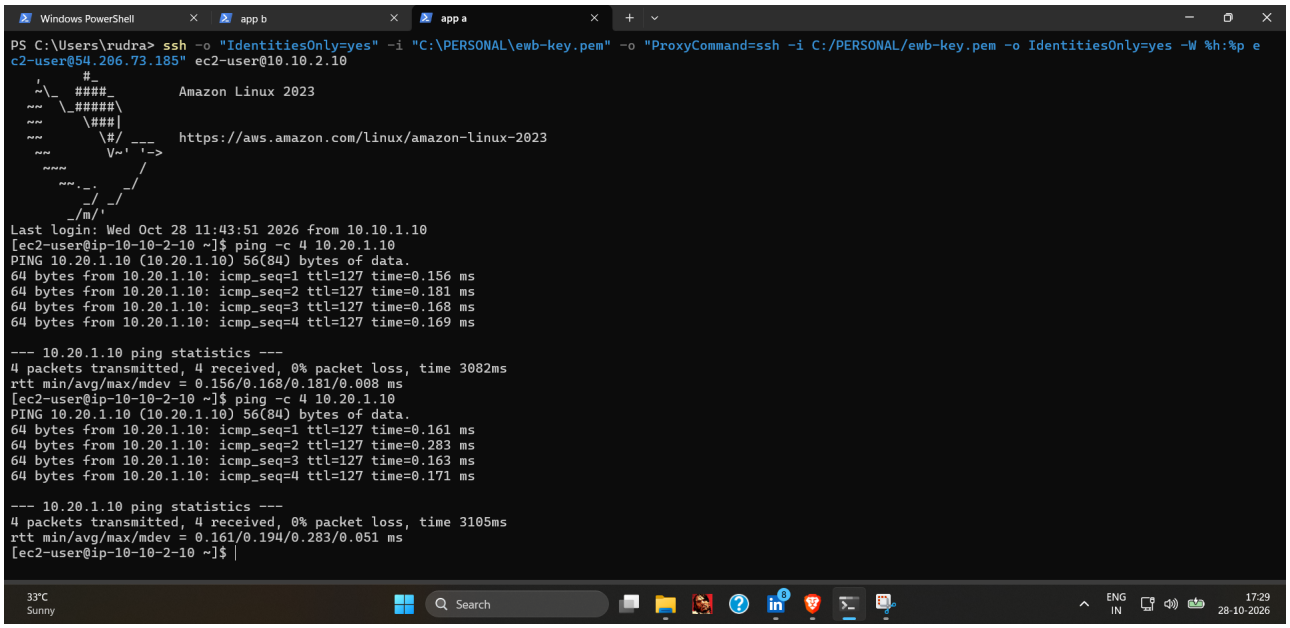


Figure 2: *Test 2: App-A to App-B ping, 4 packets received, 0% packet loss (two runs). The command line also shows SSH to App-A (10.10.2.10) through the bastion at 54.206.73.185.*

7.3 Test 3: Reverse path (App-B to App-A)

Item	Detail
Objective	Show App-B reaches App-A privately.
Method	SSH to App-B through the bastion, then run `ping -c 4 10.10.2.10`.
Expected	4 replies, 0% packet loss.
Observed	Figure 3: 4 transmitted, 4 received, 0% packet loss, round-trip about 0.17 ms.
Conclusion	Passed. App-B can reach App-A across the peering connection.

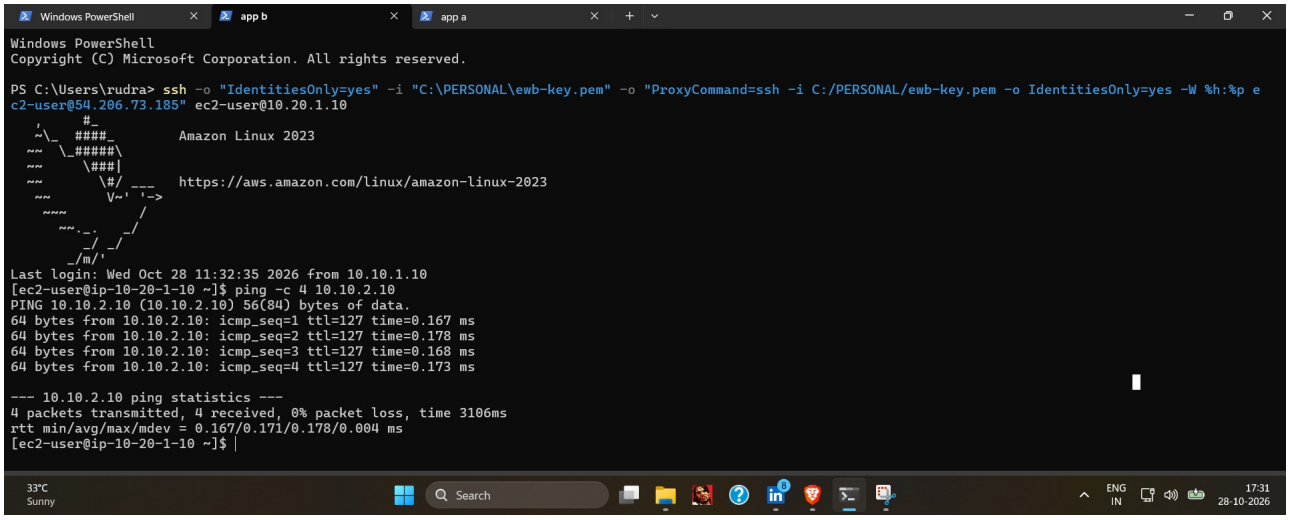


Figure 3: *Test 3: App-B (ip-10-20-1-10) to App-A ping, 4 packets received, 0% packet loss.*

7.4 Test 4: Private SSH through the bastion

Item	Detail
Objective	Show a private instance in the other VPC can be administered through the bastion using its private IP.

Item	Detail
Method	From the laptop, SSH to `ec2-user@10.20.1.10` with a ProxyCommand that hops through `ec2-user@54.206.73.185`; then run `hostname` and `hostname -I`.
Expected	A shell on App-B reporting its private address `10.20.1.10`.
Observed	Figure 4: hostname `ip-10-20-1-10.ap-southeast-2.compute.internal` and address `10.20.1.10`. The login banner shows Amazon Linux 2023 and a last login from `10.10.1.10` (the bastion).
Conclusion	Passed. App-B is reachable only through the bastion over private addressing. (The key file path appears in the command line; no key contents are shown.)

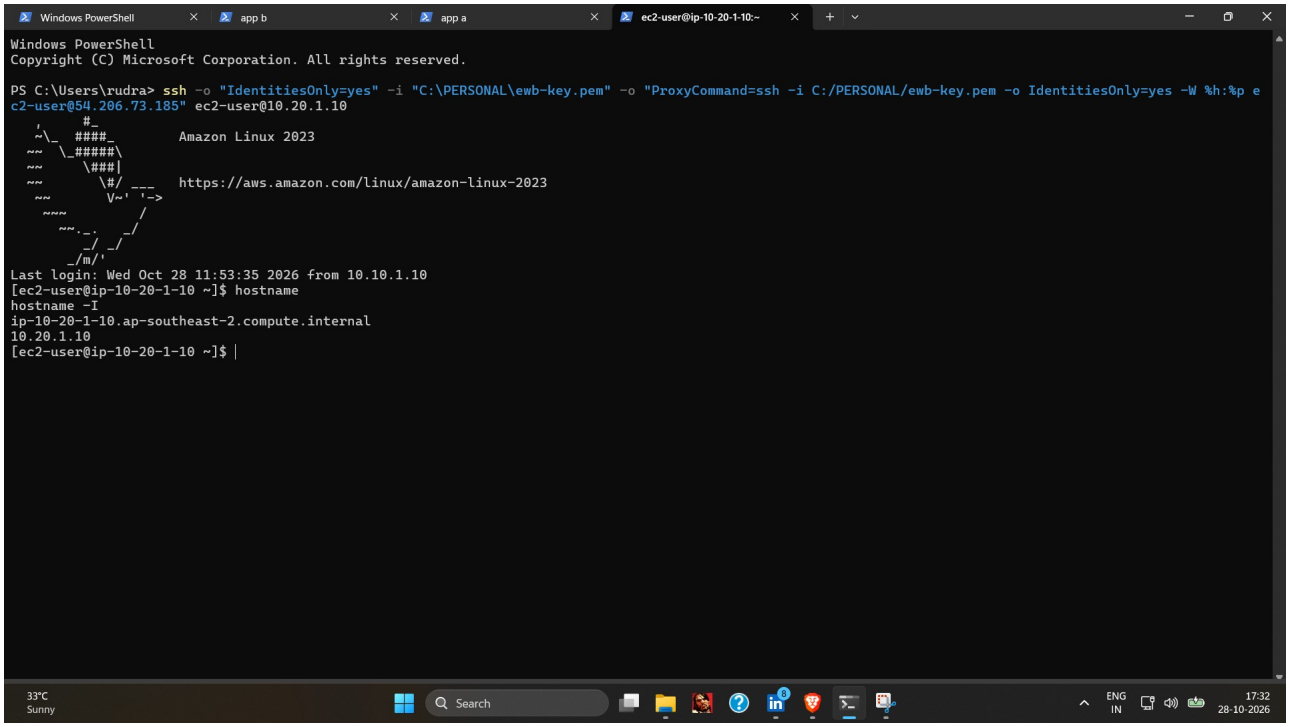


Figure 4: *Test 4: SSH to App-B (10.20.1.10) through the bastion; hostname and hostname -I confirm 10.20.1.10.*

7.5 Test 5: Isolation (no public IPv4)

Item	Detail
Objective	Verify App-A and App-B have no public IPv4 address.
Method	EC2 console, Instances, Details tab for each instance.
Expected	Public IPv4 address and Public DNS blank for both.
Observed	Figure 5 (App-B): Public IPv4 address "–", Public DNS "–", private IPv4 `10.20.1.10`. Figure 6 (App-A): the same, with private IPv4 `10.10.2.10`. Both are Running with 3/3 checks passed.
Conclusion	Passed. Neither private instance has a public IPv4 address. (The "Elastic IP" field is below the visible area in these screenshots, so it is not claimed.)

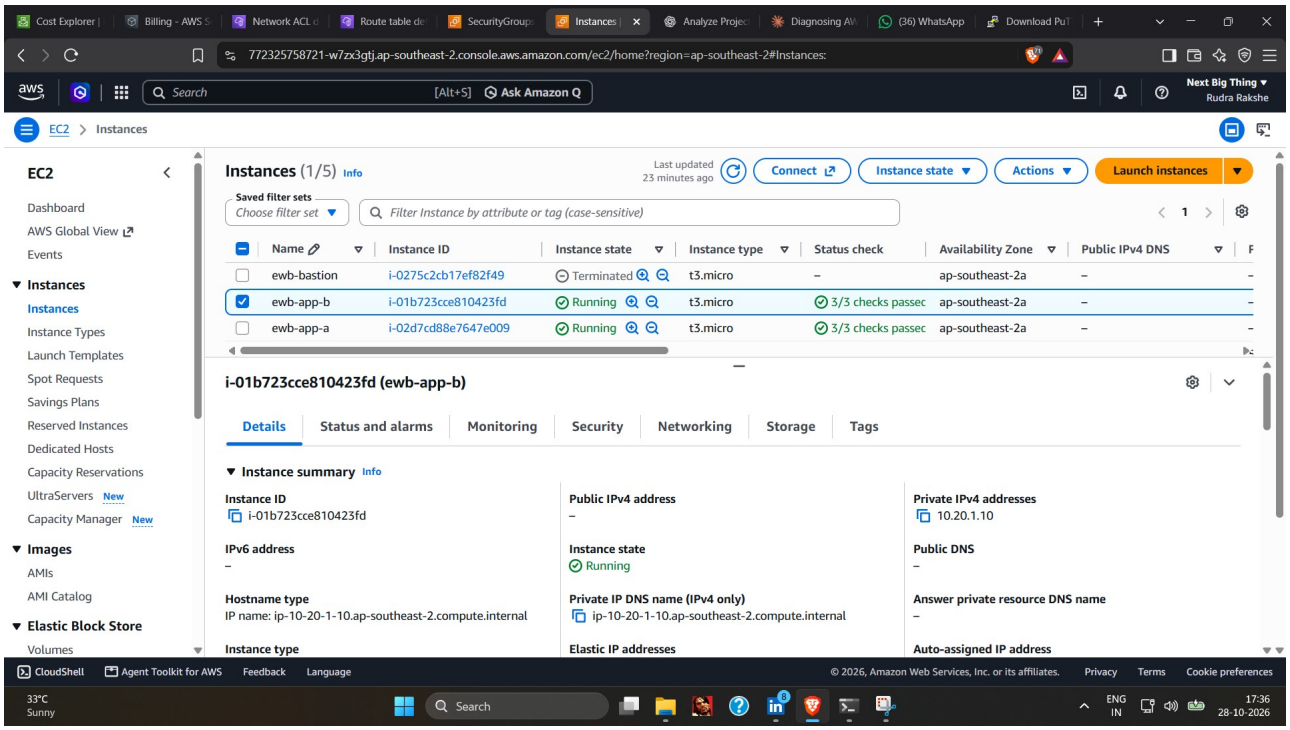


Figure 5: *Test 5: ewb-app-b (i-01b723cce810423fd) shows no Public IPv4 address and private IPv4 10.20.1.10.*

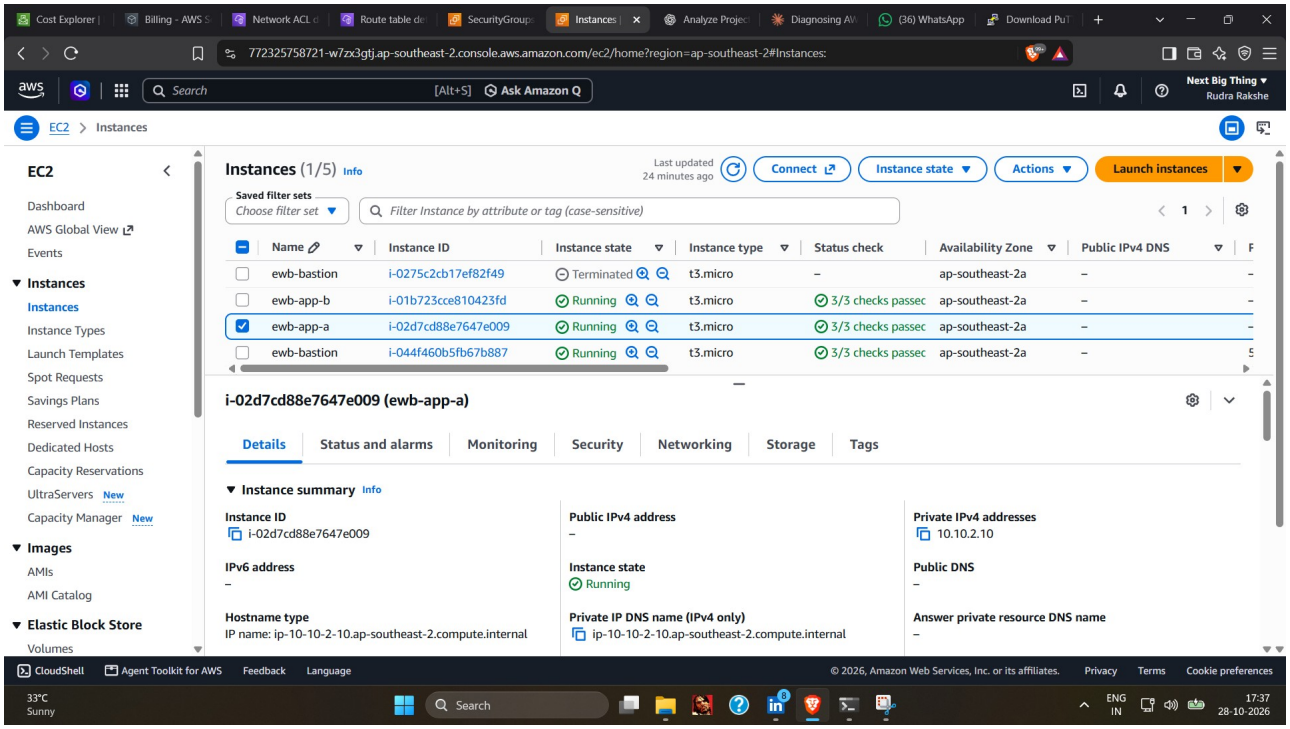


Figure 6: *Test 5: ewb-app-a (i-02d7cd88e7647e009) shows no Public IPv4 address and private IPv4 10.10.2.10.*

8. Controlled ICMP failure and recovery

This test shows that a security-group rule, not just routing, decides whether App-A's ping reaches App-B.

Step	Detail
Normal state	App-A pings App-B successfully (Test 2).

Step	Detail
Failure	Remove the inbound rule on `ewb-sg-app-b` that allows ICMP Echo Request from `10.10.2.10/32`, then ping from App-A. Expected: 100% packet loss.
Recovery	Re-add the rule (Custom ICMP, Echo Request, source `10.10.2.10/32`), then ping again from App-A. Expected: 0% packet loss.

8.1 Observed results

Failure. Figure 7 shows a successful run (4 of 4 received) followed by two runs with **4 packets transmitted, 0 received, 100% packet loss**. The terminal then reports a dropped connection (`client_loop: send disconnect: Connection reset`).

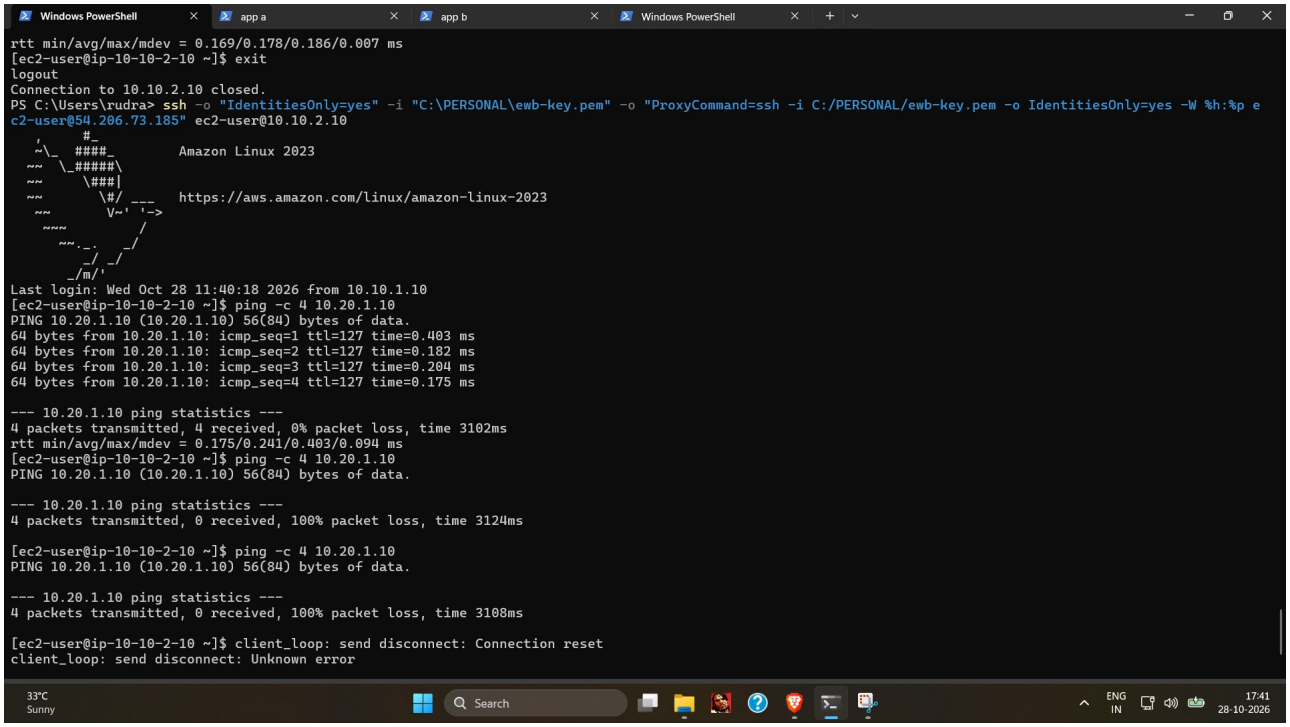


Figure 7: *Controlled failure: App-A (10.10.2.10) ping to App-B (10.20.1.10) shows 100% packet loss (two runs), after an earlier successful run in the same terminal.*

Recovery. Figure 8 shows a new session on App-A with 4 transmitted, 4 received, **0% packet loss**.

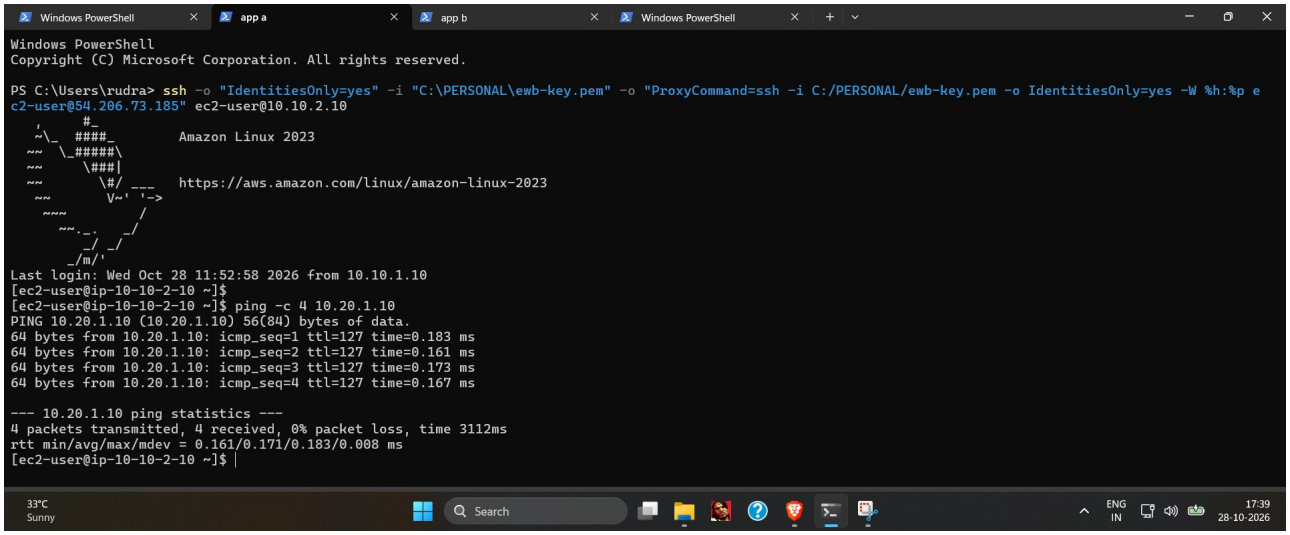


Figure 8: *Recovery: App-A ping to App-B again shows 4 packets received, 0% packet loss.*

8.2 Limits of this evidence

- **Rule changes are not captured.** No screenshot shows the ICMP rule deleted or re-added in the console, so the link between the rule and the result rests on the student's account rather than on a screenshot.
- **Order is not time-stamped.** The failure screenshot was taken at 17:41 and the recovery screenshot at 17:39 (taskbar time). The failure terminal is a scrollback of an older session, so the ping runs happened earlier than the screenshot. The ping runs themselves carry no timestamps.
- **An earlier anomaly.** After the rule appeared to be removed, pings from App-A to App-B kept succeeding for a time. Pings from App-B to App-A are expected to succeed, because App-B starts that connection and its outbound rules allow it; the deleted rule only governs traffic **initiated by** App-A. The supplied screenshots do not record what was changed before the failure in Figure 7 appeared, so this report does not name a cause.

8.3 What this demonstrates

Because routes, peering and network ACLs were unchanged, the loss of replies after the rule was removed, and their return afterwards, is consistent with the inbound ICMP rule on `ewb-sg-app-b` being what permits App-A's pings. Stating it as proven would need the missing console screenshots listed in Appendix A.

9. Troubleshooting and lessons learned

Problem	What was found	Resolution / lesson
SSH to the bastion refused or timing out	The original bastion (`i-0275c2cb17ef82f49`) was unusable and was replaced. The replacement received a new public IP (`54.206.73.185`); testing the old IP (`3.106.134.175`) gave "Connection refused" because the address no longer belonged to the project. After that, the bastion's security group did not match the laptop's public IP (mobile hotspot).	Always re-read the instance's current public IPv4. With a changing hotspot IP, a temporary open rule confirmed the diagnosis (reported by the student). Narrow the rule back to a `/32` afterwards.
"Connection refused" vs timeout	A security group or ACL block normally causes a timeout (packets dropped). A refusal means something answered with a reset, such as a closed port or an address that belongs to someone else.	Distinguish the two before changing rules.
`ssh -J` rejected at the bastion	`-J` did not pass the key to the jump host.	A ProxyCommand with `-i` on both hops worked (visible in Figures 2 to 4).
`ping -c` failed in PowerShell	The command was run on the Windows laptop, where `-c` is not a valid ping option.	Run Linux ping inside the EC2 shell (prompt `[ec2-user@ip-10-10-2-10 ~]$`).
Ping kept working after an ICMP rule was removed	B-to-A pings are not governed by the deleted rule (stateful behaviour). The A-to-B anomaly is described in section 8.2.	Test in the correct direction, start a new ping after each change, and screenshot the rule before and after.

Lessons learned

- Security groups are stateful: an inbound rule matters only for connections started toward that instance.
- Document each rule change with a console screenshot and a timestamped terminal run, so the failure and recovery sequence can be proven.

- A public IPv4 address that is not an Elastic IP changes when an instance is replaced or stopped and started.
- Do not leave SSH open to `0.0.0.0/0`.

10. Cleanup evidence and cost considerations

Resource	Evidence	Status
EC2 instances (App-B, App-A, bastion)	Figure 9: all three shown as Terminated (17:51). The original bastion was already terminated earlier (visible in Figure 5 and Figure 6 instance lists).	Verified
EBS volumes	Figure 10: "No matching volumes found" in ap-southeast-2 (17:52).	Verified
VPC peering connection	Figure 11: "successfully deleted peering connection pcx-04950eee78a68adf1", status Deleted (17:56).	Verified
Elastic IPs	None known to have been allocated; no screenshot supplied.	Pending: not evidenced
Route tables, subnets, internet gateway, VPCs, security groups	No screenshots supplied. Route entries that pointed at the deleted peering connection show as "blackhole" if their route tables still exist.	Pending: not evidenced
Bastion SSH rule (`0.0.0.0/0`)	Final state not shown; the bastion is terminated.	Pending: not evidenced

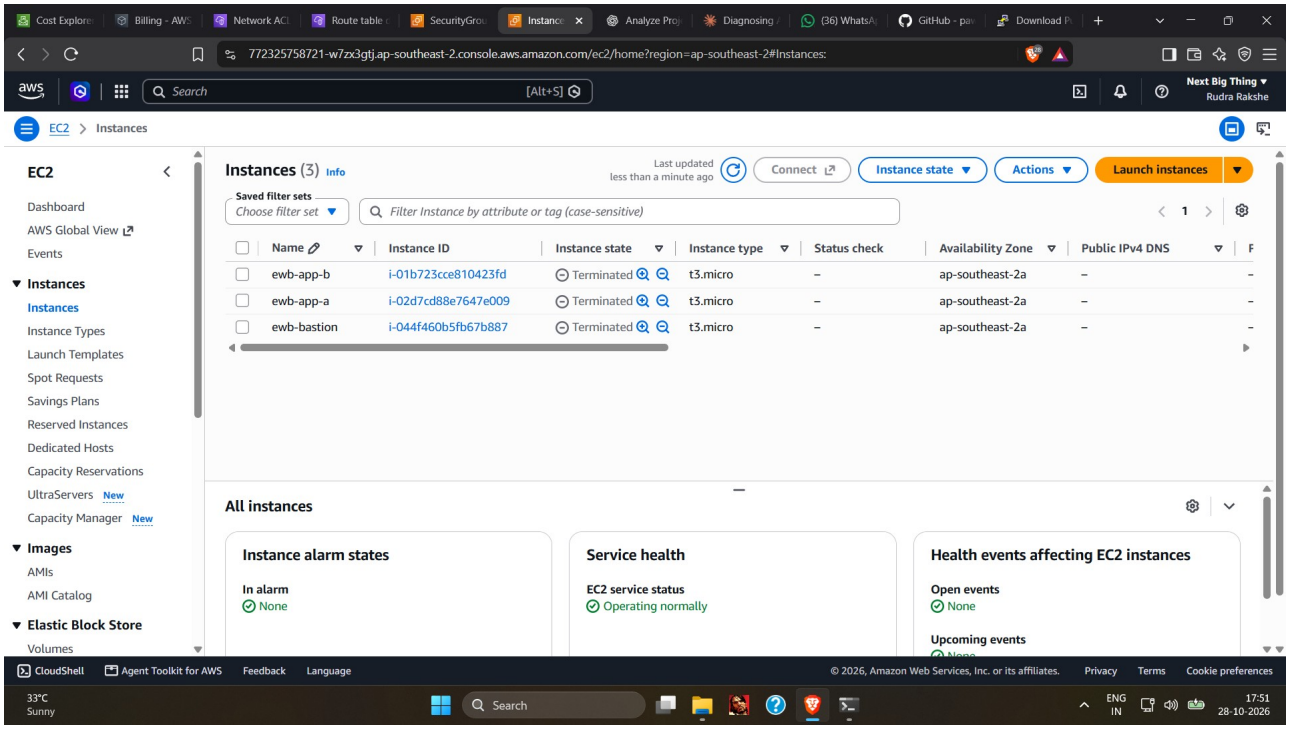


Figure 9: *Cleanup: ewb-app-b, ewb-app-a and ewb-bastion (i-044f460b5fb67b887) all show Terminated.*

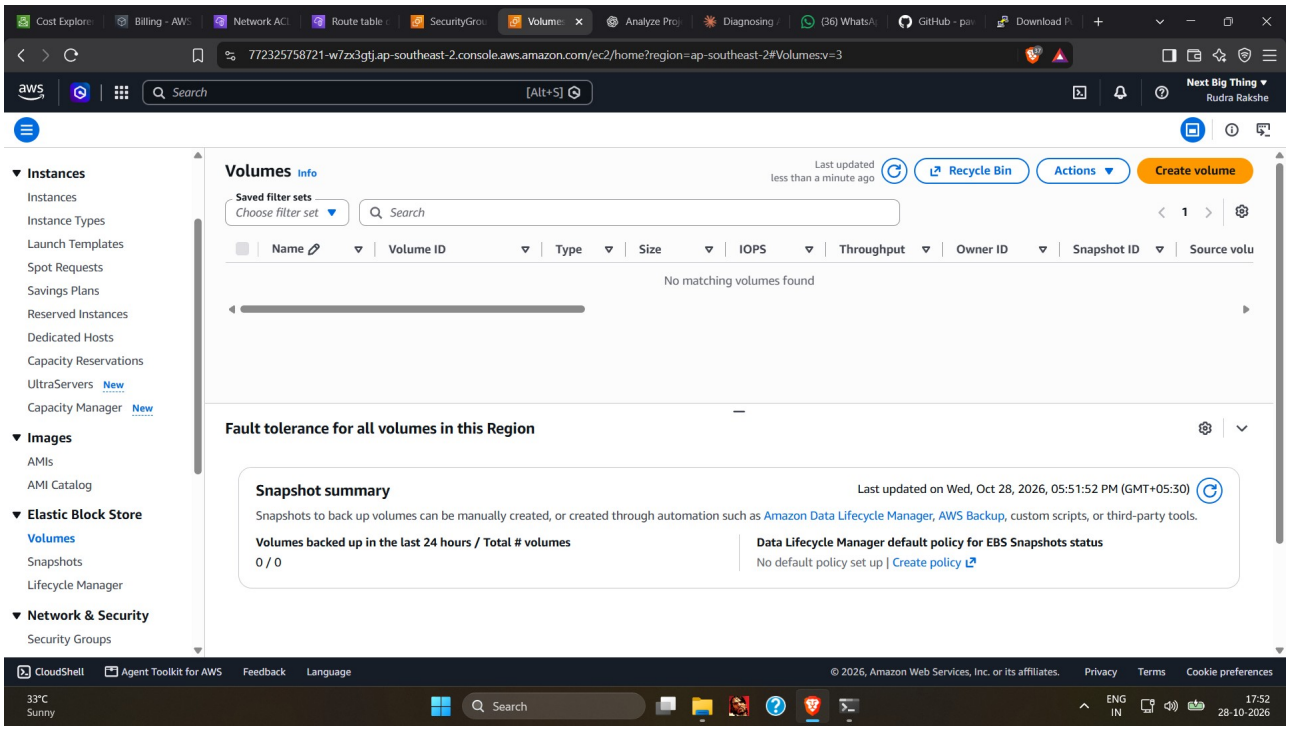


Figure 10: *Cleanup: Volumes page in ap-southeast-2 shows "No matching volumes found".*

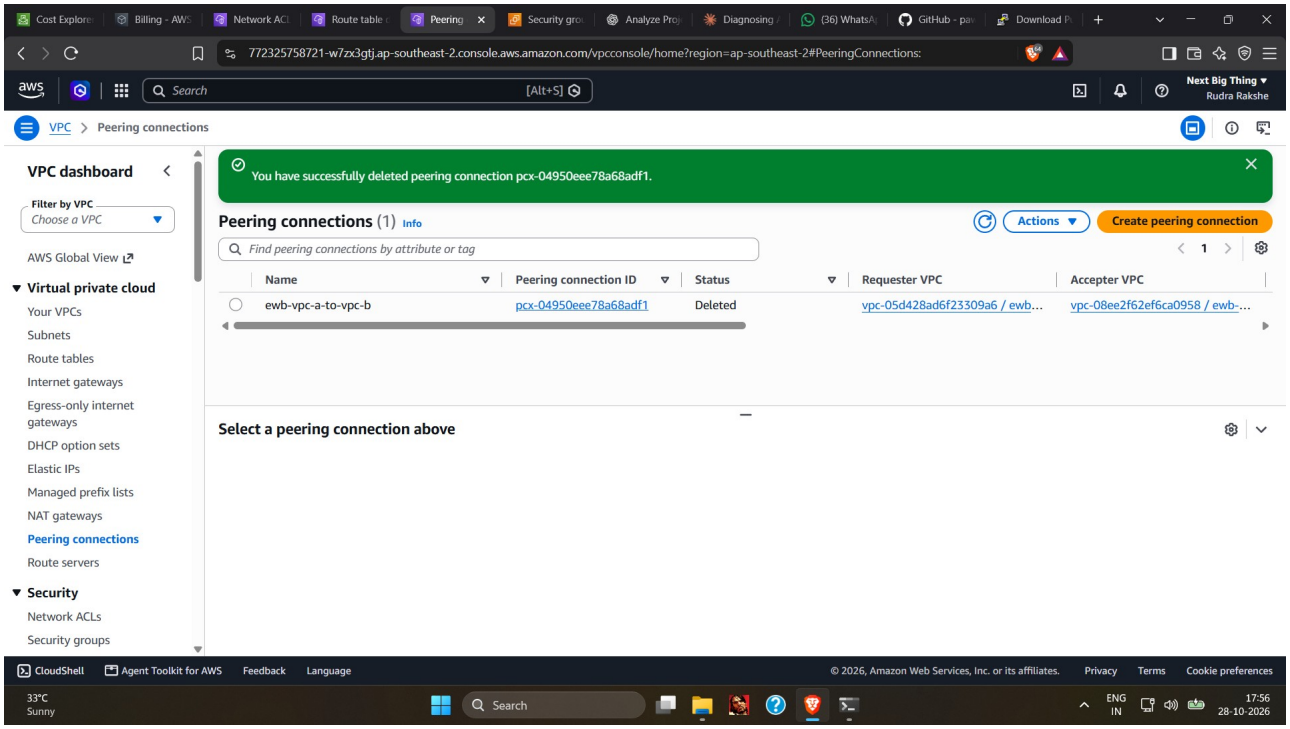


Figure 11: *Cleanup: peering connection pcx-04950eee78a68adf1 (ewb-vpc-a-to-vpc-b) deleted.*

Cost considerations. EC2 instances, attached EBS volumes and public IPv4 addresses can incur charges, including Elastic IPs that are allocated but not in use. Charges stop when instances are terminated and their volumes deleted. Re-check Instances, Volumes and Elastic IPs in **ap-southeast-2** (Sydney), since resources in another region are not shown. VPCs, subnets, route tables and internet gateways have no hourly charge, but delete them if the project is finished. No NAT Gateway was used.

11. Conclusion

The two VPCs, connected by `pcx-04950eee78a68adf1`, communicated privately in both directions (Tests 2 and 3), the private instances were reached only through the bastion (Test 4), and neither had a public IPv4 address (Test 5). A removed ICMP rule coincided with 100% packet loss and its restoration with recovery, although the rule edits themselves were not captured. Test 1 could not be verified, and cleanup is evidenced for instances, volumes and the peering connection only.

The main technical lessons are the difference between a refusal and a timeout, the role of stateful security groups, and the cost of an unstable public IP when restricting SSH.

12. Appendix

Appendix A: Missing evidence and checklist

Item	Status / what would close the gap
Test 1 baseline failure	Evidence not provided. Would need a ping with 100% loss before peering and routes existed.
Security-group rule removal and restoration	Not captured. Add console screenshots of `ewb-sg-app-b` inbound rules without and with the ICMP rule.
Time-ordered failure then recovery	Not shown. Re-run both with `date; ping -c 4 -W 2 10.20.1.10` if the resources still existed.
VPC, subnet, route-table, internet-gateway and security-group screenshots	Not provided. Resource details come from supplied text.
Cleanup of VPCs, subnets, route tables, IGW, security groups, Elastic IPs	Not evidenced.
Project PDF submission checklist	Could not be checked; the PDF was not available.

Appendix B: Supplementary file test1.png

This is the file supplied for Test 1. It shows a successful ping with peering active, so it is not a baseline failure.

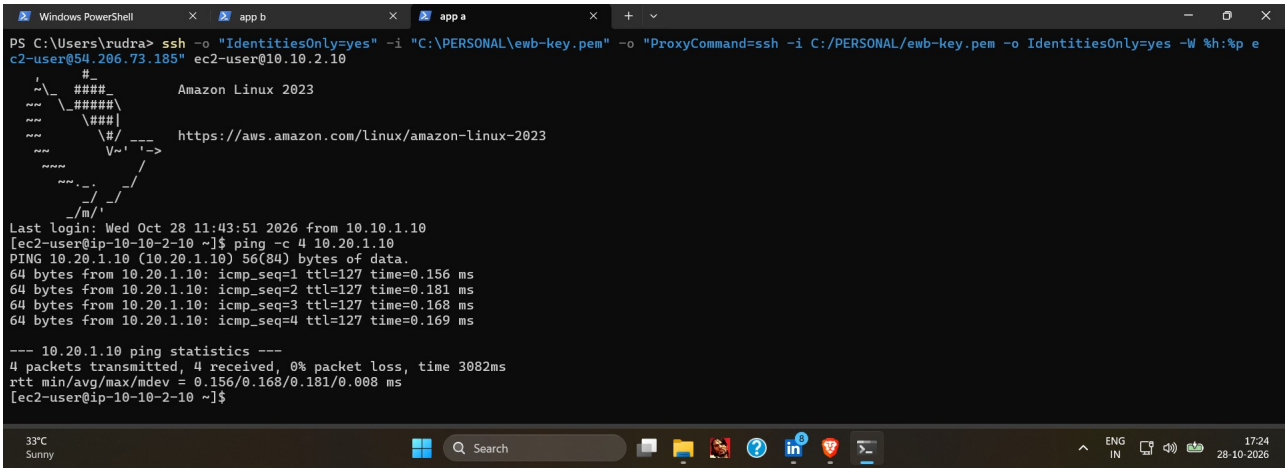


Figure B1: *Supplementary: first App-A to App-B ping, 4 packets received, 0% packet loss (peering active, not a baseline).*

Appendix C: Evidence file map

Supplied file	Used as
`test1.png`	Figure B1 (supplementary)
`test2-forwardping.png`	Figure 2
`test3-reverping.png`	Figure 3
`test4-privatessh_test.png`	Figure 4
`test5p1_isolation.png`	Figure 5
`test5p2.png`	Figure 6
`test6_failure.png`	Figure 7
`test6_p2_recovery.png`	Figure 8
`cleanupinstances.png` (same image as `Screenshot_2026-10-28_175128.png`)	Figure 9
`volume_cleanup.png` (same image as `Screenshot_2026-10-28_175214.png`)	Figure 10
`Screenshot_2026-10-28_175625.png`	Figure 11